Assignment#02

Full-Stack Food Delivery System

Course: Mobile Application Development

By

Rukhsar Jabeen(8961)

Submitted to: Ma,am Sadaf

Subject: MAD

Date:24-03-2026

Batch:Spring-2024

Semester:Fifth

DEPARTMENT OF COMPUTER SCIENCE

ABASYN UNIVERSITY

ISLAMABAD CAMPUS

1. Introduction:

- The purpose of this assignment is to demonstrate the communication between a React Native (Frontend) application and a Node.js/Express (Backend) server. The project focuses on two main functionalities:
- Fetching and displaying a product menu from a local server.
- Capturing user orders from the mobile interface and logging them into the server's terminal for administrative tracking.

2-System Specifications & Tech Stack:

This project was built using modern web and mobile development standards. The specific technologies utilized include:

Frontend Environment: React Native (using Expo CLI for rapid development and testing).

Backend Runtime: Node.js (providing a JavaScript execution environment outside the browser).

Server Framework: Express.js (used to create efficient RESTful API routes like /api/inventory and /place-order).

Network Protocol: HTTP/REST (using GET for fetching the menu and POST for submitting orders).

Environment Requirement: Both the React Native client and the Node.js server must be operating on the same Local Area Network (LAN) / Localhost IP for successful communication.

3-Port Configuration:

Port 6060: Dedicated to the Menu Inventory API (Final_menu.js).

Port 5050 / 8080: Utilized for the Order Logging Service (server.js and order_logger.js).

Data Protocol: RESTful API using JSON for request and response payloads.

Backend API Development (Menu Hosting)

4-Implementation (Final_menu.js):

The Final_menu.js script is the foundation of the system's data architecture. It performs the following key functions:

- **JSON Data Hosting:** It stores a structured list of food items (ID, Name, Price, Image) and serves it as a JSON API. This allows the app to fetch real-time data instead of using static text.
- **Endpoint Creation:** It defines a specific GET route (/api/inventory) on Port 6060, which acts as the "address" where the mobile app requests the menu.
- **CORS Enabling:** The script uses CORS middleware to grant permission to the Expo Browser to access the backend data securely without being blocked by browser security policies.
- **External Image Integration:** It maps product IDs to high-quality Unsplash URLs, ensuring the app can display professional food visuals without increasing the server's storage load.

4.1-Server Side logic:

- **Initialization:** The script uses the express() application factory.
- **Middleware:** Integrated cors() to ensure that the Expo Browser can bypass security policies and fetch the data successfully.
- **Port Binding:** The service is bound to a specific listener, making the menu globally accessible across the Local Area Network (LAN).

C: > Users > aashi > backend > Final_menu.js > productCatalog > category

```
const express = require('express');
const app = express();
app.use(express.json()); // JSON parsing support

const PORT = 6060;

// --- Q1 & Q2: THE COMPLETE PRODUCT CATALOG (Menu Database) ---
// Har item ke saath Image URL (Unsplash) zaroor hai
const productCatalog = [
  { id: 101, name: "Zinger Deluxe", price: 750, category: "Fast Food", img: "https://images.unsplash.com/photo-1571091718767-18b5b1457add" },
  { id: 102, name: "Grilled BBQ Burger", price: 850, category: "Fast Food", img: "https://images.unsplash.com/photo-1568901346375-23c9450c58cd" },
  { id: 103, name: "Loaded Cheese Fries", price: 450, category: "Fast Food", img: "https://images.unsplash.com/photo-1585109649139-366815a0d713" },
  { id: 104, name: "Mutton Biryani Special", price: 950, category: "Daily Specials", img: "https://images.unsplash.com/photo-1589302168068-964664d
  { id: 105, name: "Garlic Naan (Butter)", price: 80, category: "Daily Specials", img: "https://images.unsplash.com/photo-1626074353765-517a681e49
  { id: 106, name: "Midnight Cold Coffee", price: 550, category: "Drinks", img: "https://images.unsplash.com/photo-1517701604599-bb29b565096c" },
  { id: 107, name: "Fresh Mint Lemonade", price: 250, category: "Drinks", img: "https://images.unsplash.com/photo-1513558161293-cdaf765ed2fd" },
  { id: 108, name: "Chocolate Lava Cake", price: 400, category: "Desserts", img: "https://images.unsplash.com/photo-1606313564200-e75d5e304376c" },
  { id: 109, name: "Alfredo Pasta", price: 890, category: "Italian", img: "https://images.unsplash.com/photo-1645112481355-813ca3377756" },
```

localhost:6060/api/inventory

Pretty-print


```
[
  {
    "id": 101,
    "name": "Zinger Deluxe",
    "price": 750,
    "category": "Fast Food",
    "img": "https://images.unsplash.com/photo-1571091718767-18b5b1457add"
  },
  {
    "id": 102,
    "name": "Grilled BBQ Burger",
    "price": 850,
    "category": "Fast Food",
    "img": "https://images.unsplash.com/photo-1568901346375-23c9450c58cd"
  },
  {
    "id": 103,
    "name": "Loaded Cheese Fries",
    "price": 450,
    "category": "Fast Food",
    "img": "https://images.unsplash.com/photo-1585109649139-366815a0d713"
  },
  {
    "id": 104,
    "name": "Mutton Biryani Special",
    "price": 950,
    "category": "Daily Specials",
    "img": "https://images.unsplash.com/photo-1589302168068-964664d93dc0"
  },
  {
    "id": 105,
    "name": "Garlic Naan (Butter)",
    "price": 80,
    "category": "Daily Specials",
    "img": "https://images.unsplash.com/photo-1626074353765-517a681e40be"
  },
```

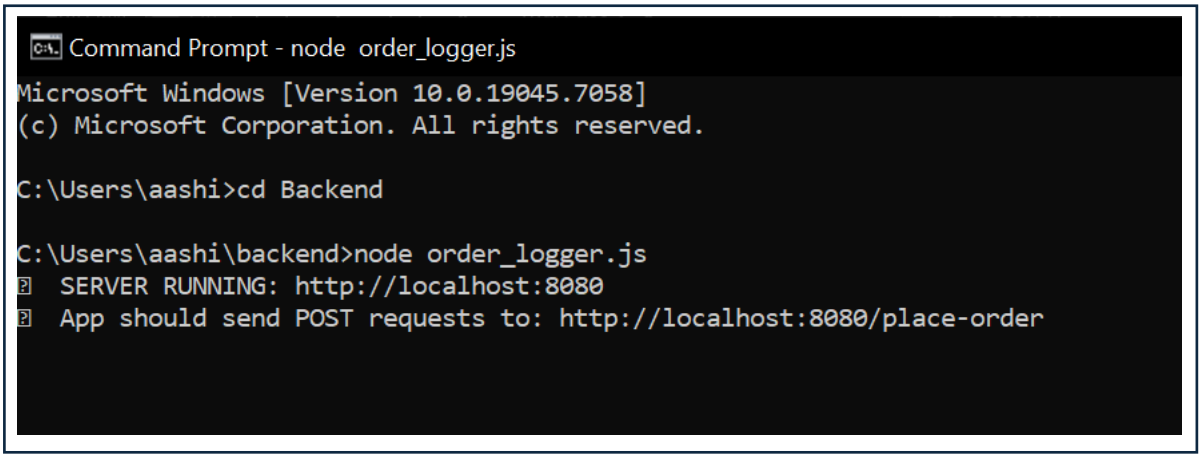


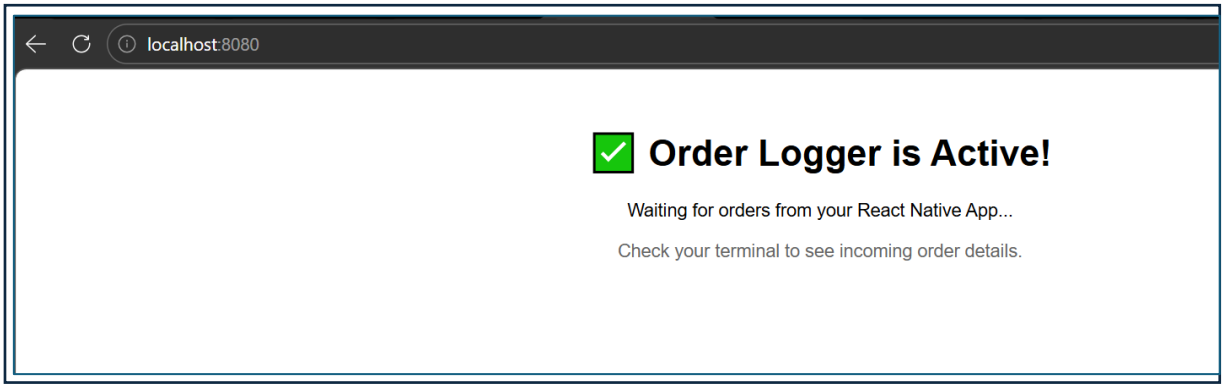
```
  {
    "id": 106,
    "name": "Midnight Cold Coffee",
    "price": 550,
    "category": "Drinks",
    "img": "https://images.unsplash.com/photo-1517701604599-bb29b565090c"
  },
  {
    "id": 107,
    "name": "Fresh Mint Lemonade",
    "price": 250,
    "category": "Drinks",
    "img": "https://images.unsplash.com/photo-1513558161293-cdaf765ed2fd"
  },
  {
    "id": 108,
    "name": "Chocolate Lava Cake",
    "price": 400,
    "category": "Desserts",
    "img": "https://images.unsplash.com/photo-1606313564200-e75d5e30476c"
  },
  {
    "id": 109,
    "name": "Alfredo Pasta",
    "price": 890,
    "category": "Italian",
    "img": "https://images.unsplash.com/photo-1645112481355-813ca3377756"
  },
  {
    "id": 110,
    "name": "Steak with Mash",
    "price": 1450,
    "category": "Main Course",
    "img": "https://images.unsplash.com/photo-1600891964599-f61ba0e24092"
  }
]
```

5-Backend Status Validation (order_logger.js):

The order_logger.js script serves as the Verification Layer for the backend. Before any transaction occurs, it ensures the communication channel is open.

- **Service Readiness Indicator:** This script provides a visual confirmation by displaying a "Ready!" message in the web browser. This acts as a diagnostic tool to ensure the Node.js environment is active.
- **Endpoint Listening:** It is configured to listen on a specific Network Port (8080). This creates a dedicated "Gateway" that waits specifically for incoming data packets from the Expo Browser.
- **Request Routing:** It defines the /place-order endpoint. This ensures that when the frontend sends a POST request, the backend knows exactly which function should handle the incoming customer data.
- **Terminal Integration:** The logger is directly linked to the VS Code Terminal. Instead of saving data to a file, it uses console.log() to provide an instantaneous, real-time display of the order payload for the administrator.

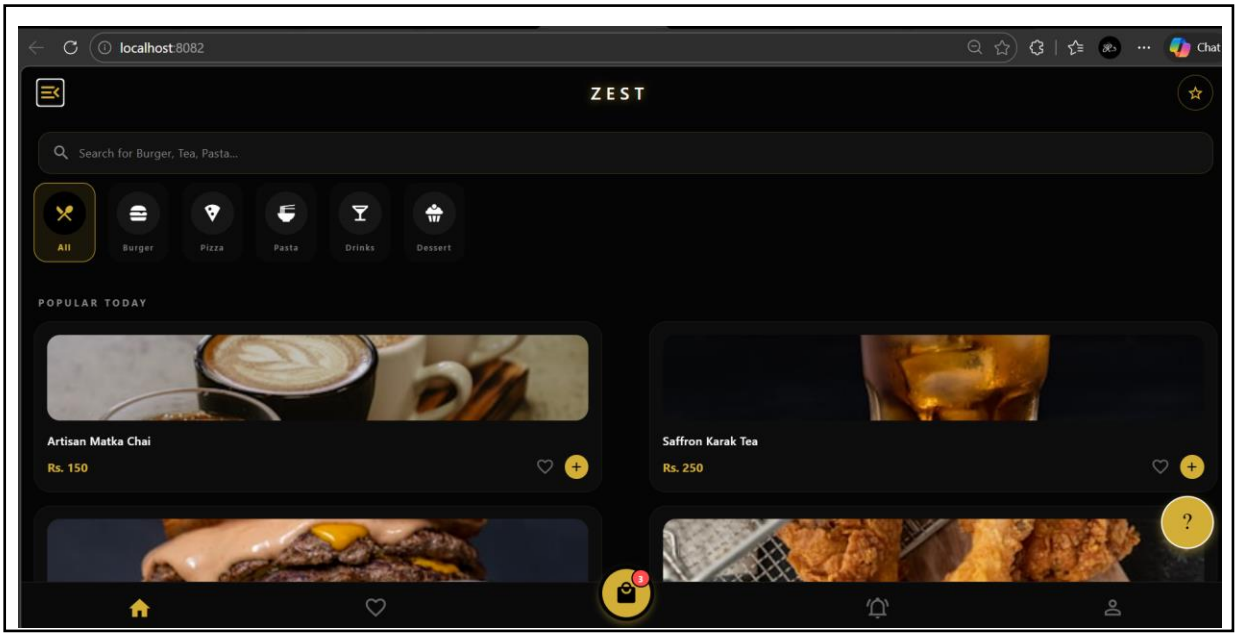




6-Real-time Order Logging (server.js + Expo Browser):

This phase demonstrates the successful Full-Stack Integration between the frontend interaction and the backend response.

- **POST Endpoint Execution:** The server.js script provides a dedicated POST route (/place-order), allowing the Expo Browser to securely transmit customer cart data and total prices to the server.
- **JSON Payload Parsing:** Using express.json() middleware, the backend automatically translates raw data from the browser into a readable format for terminal output.
- **Live Terminal Monitoring:** The core function is Instant Logging. Once "Confirm Order" is triggered, the backend uses console.log() to display the full order details directly in the VS Code Terminal.
- **Response Handshake:** The backend sends a 200 OK status back to the Expo Browser, confirming that the data has been received and processed by the administrative console.



```
Select Command Prompt - node server.js

C:\Users\aashi\backend>node server.js
Server is running: http://localhost:5050
Waiting for orders from your React Native app...
Raw Data Received: {
  orderId: '#2918',
  itemsList: 'Artisan Matka Chai, Wagyu Beef Slider, Saffron Karak Tea',
  totalBill: 1400,
  deliveryLocation: 'house  4'
}

********** NEW ORDER RECEIVED **********
Order ID:  #2918
Items:     Artisan Matka Chai, Wagyu Beef Slider, Saffron Karak Tea
Total:     Rs. 1400
Address:   house  4
****************************************

Raw Data Received: {
  orderId: '#7153',
  itemsList: 'Artisan Matka Chai, Wagyu Beef Slider, Crispy Zinger, Woodfire Pepperoni',
  totalBill: 2900,
  deliveryLocation: 'house 5'
}
```

6.1:Frontend-Backend Integration (The Communication Protocol):

To establish a functional communication bridge, the React Native frontend utilizes the native fetch() API within the executeFinalCheckout function. The following protocols were implemented:

Network Bridging: Both the Node.js server and the Expo Browser were connected to the same Local Area Network (LAN) to allow seamless data packet discovery.

Static IP Addressing: To bypass localhost limitations on external browsers, a specific Internal IP (192.168.1.109) was targeted to map directly to Port 8082.

Asynchronous Logic: The implementation uses an async/await pattern to handle network latency, ensuring the Expo UI remains responsive while data is transmitted to the terminal.

```
        await fetch('http://localhost:5050/place-order', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(orderPayload),
        });
        console.log("Data Sent Successfully!");

      } catch (error) {
```

7- Core Features:

7.1- Dynamic Menu Retrieval:

- Menu data is fetched from the backend instead of being hardcoded
- Implemented using the useEffect hook
- Ensures real-time updates from the server

7.2-Asynchronous Order Handling:

- Orders are sent using asynchronous POST requests
- Prevents UI blocking during network operations
- Enhances user experience

7.3 Multi-Service Communication:

- Separate backend services handle:
  - Data retrieval (Menu Service)
  - Transaction logging (Order Service)
- Enables modular and scalable system design

8- System Workflow:

Component	Port	Method	Action	Data Flow
Mobile App (Client)	N/A	HTTP GET	Request Menu	/menu request
Menu Server	6060	HTTP	Send Menu Data	JSON response
Mobile App (Client)	N/A	HTTP POST	Place Order	Order JSON payload
Order Logger	8080	HTTP	Log Order	Status: 200 OK
Terminal (Monitor)	N/A	Console	Display Logs	“New Order Received!”

9-Conclusion:

This project successfully connects a React Native (Expo) frontend with a Node.js backend. It proves that we can send and receive data in real-time between a mobile interface and a server terminal.

Key Achievements:

Full-Stack Connectivity: We moved data from the Expo Browser to the VS Code Terminal using a professional API setup.

Smart Port Management: We effectively used different ports (8082, 5050, and 8080) to handle the menu and order details separately.

Reliable Logic: By using async/await and Static IPs, we ensured the app stays fast and the connection remains stable.